Case Study 8: Text Compression Using Character Runs

Assignment Case No. 35 | Level: Beginner

01 PROBLEM STATEMENT

Encode a string by grouping consecutive repeated characters and compare original and encoded lengths, including cases where encoding is not beneficial.

Problem interpretation: Run-length encoding replaces each consecutive group of the same character with the character followed by its count. If the encoded version is not shorter, keep the original string.

02 OBJECTIVES

Identify consecutive repeated characters.

Create a run-length encoded string.

Compare original and encoded lengths.

Keep the original when compression does not save space.

03 ASSUMPTIONS

The input is a non-empty string for this example.

Characters are case-sensitive.

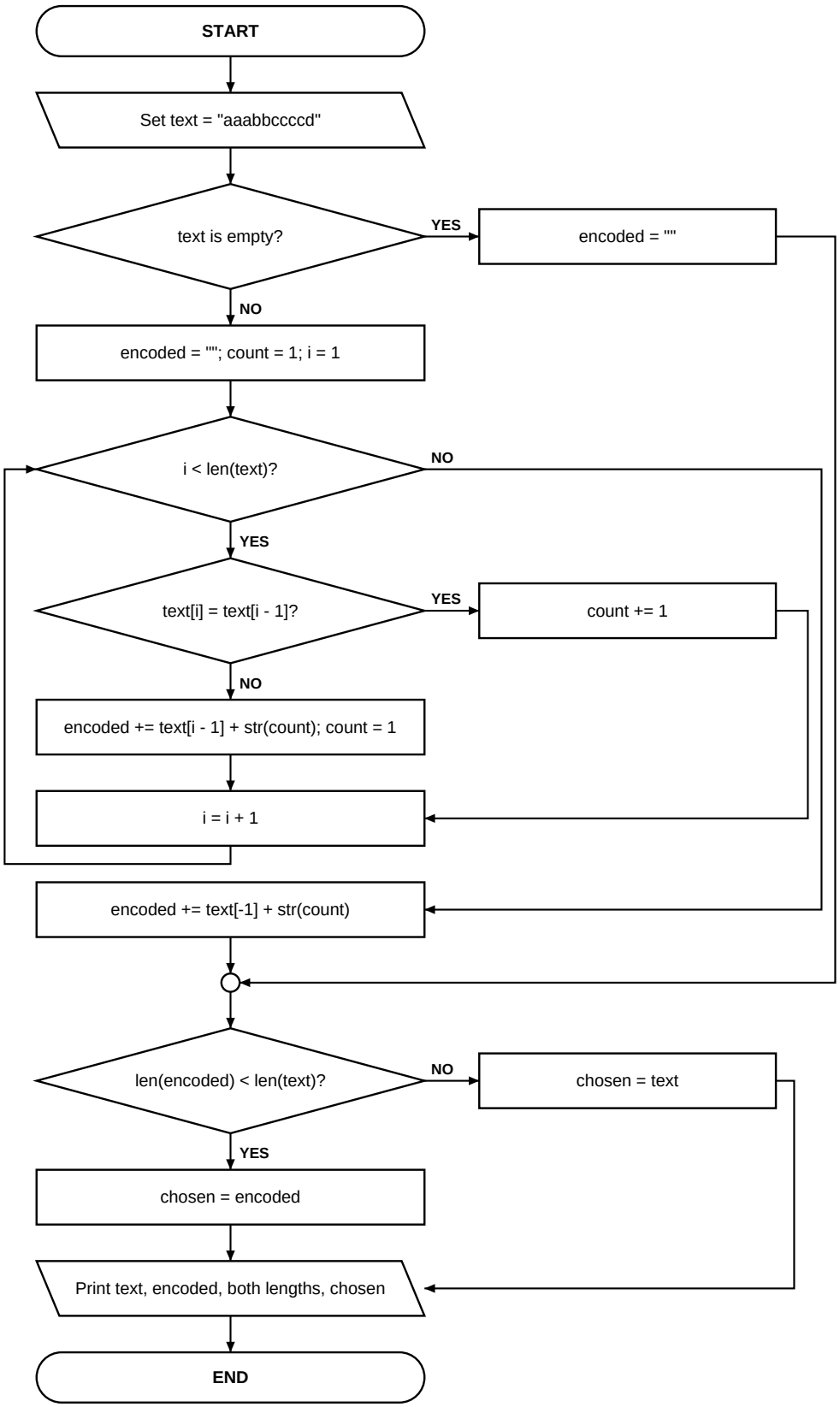
The encoded format is character followed by count, including count 1.

The program selects the shorter representation.

04 ALGORITHM

1. Start with a sample string.
2. If the string is empty, handle it separately.
3. Track the current character and its count.
4. When the character changes, append the previous character and count.
5. Append the final character run.
6. Compare encoded and original lengths.
7. Display both and the chosen representation.
8. Stop.

05 FLOWCHART (TEXT FLOW)



06 PSEUDOCODE

```
BEGIN
    text = "aaabbccccd"
    IF text is empty THEN
        encoded = ""
    ELSE
        encoded = ""; count = 1
        FOR i = 1 TO length(text) - 1
            IF text[i] = text[i-1] THEN
                count = count + 1
            ELSE
                encoded = encoded + text[i-1] + count
                count = 1
            END IF
        END FOR
        encoded = encoded + last character + count
    END IF
    IF length(encoded) < length(text) THEN
        chosen = encoded
    ELSE
        chosen = text
    END IF
    PRINT text, encoded, lengths, chosen
END
```

07 PYTHON SOURCE CODE

The complete Python program is provided in the separate file 08_Text_Compression_Using_Character_Runs_Code.pdf.

08 TEST CASES

INPUT / SCENARIO	CHECK	EXPECTED RESULT
aaabbccccd	Encoded a3b2c4d1 (8) is shorter than 10	Encoded form chosen
abcd	Encoded a1b1c1d1 (8) is longer than 4	Original form kept
xxxx	Encoded x4 (2) is shorter than 4	Encoded form chosen
(empty string)	Encoded length 0 is not less than 0	Original (empty) kept

09 SAMPLE OUTPUT PREVIEW

```
Original text: aaabbccccd
Encoded text: a3b2c4d1
Original length: 10
Encoded length: 8
Chosen representation: a3b2c4d1
```

10 CONCLUSION

The program demonstrates string indexing, loops, counters, and a practical comparison of original and encoded lengths.